

Computer Vision (CAP4410) Assignment 4

Lukas Kelk

Computer Engineer

Department of Computer Science
Florida Polytechnic University, Florida, USA

April 2025

CONTENTS

CONTENTS	ii
LIST OF FIGURES	iii
1 INTRODUCTION	1
2 IMPLEMENTATION	2
2.1 SIFT IMPLEMENTATION	2
2.1.1 First Step: Scale-space extrema detection.	2
2.1.2 Second Step: Keypoint localization	3
2.1.3 Third Step: Orientation assignment.	4
2.1.4 Fourth Step: Keypoint descriptor.	5
2.2 RESULTS	6
2.3 DISCUSSION	9
REFERENCES	10

LIST OF FIGURES

Figure 2.1	First Step: Scale-space extrema detection	3
Figure 2.2	Second Step: Keypoint localization	4
Figure 2.3	Third Step: Orientation assignment	5
Figure 2.4	Mathching	6
Figure 2.5	Test 1	6
Figure 2.6	Test 2	7
Figure 2.7	Test 2	8

1 INTRODUCTION

This report describes my implementation of the Scale-Invariant Feature Transform (SIFT) algorithm for extracting features and describing digital images. SIFT is a robust vision algorithm for salient point detection in the image that is insensitive to scaling, rotation, and illumination changes. For this project, I generated multiple versions of input images with different transformations and carried out the significant phases of the SIFT algorithm in an orderly manner: scale-space extrema detection, keypoint localization, and assignment of orientation. I monitored the algorithm's performance in different transformations of an image carefully in the process, noting down the intermediate outcomes at each step. My implementation used Python programming with the help of the OpenCV and NumPy libraries, which were found to be powerful tools for handling images as well as mathematical operations. In this report, I explain my procedural method, implementation decisions, experimental findings, and analysis of performance of the SIFT algorithm for feature detection and description in images.

2 IMPLEMENTATION

2.1 SIFT IMPLEMENTATION

2.1.1 First Step: Scale-space extrema detection

The First step in SIFT is Scale-Space extrema implementation. Using the openCV library we can actually complete all of the SIFT Steps but then separate them into the steps. OpenCV has built in SIFT functionality. Creating a SIFT object will allow us to complete the sift algorithm on a given image.

```
1 sift = cv2.SIFT_create()
2
3 kp1, des1 = sift.detectAndCompute(img1, None)
4
5 img1Extrema = img1.copy()
```

Once a SIFT Object is created we can extract all the steps from the keypoint features. To identify the Scale-Space Extrema we can simply plot the X,Y coordinates of the points to demonstrate the step of locating them

```
1 for kp in kp1:
2     x, y = kp.pt
3     cv2.circle(img1Extrema, (int(x), int(y)), 2, (0, 0, 255), 2)
4
5 imageRightPhoto = ConvertCV2ToTKINTER(img1Extrema)
6 imageRight.config(image=imageRightPhoto)
7 imageRight.image = imageRightPhoto
```

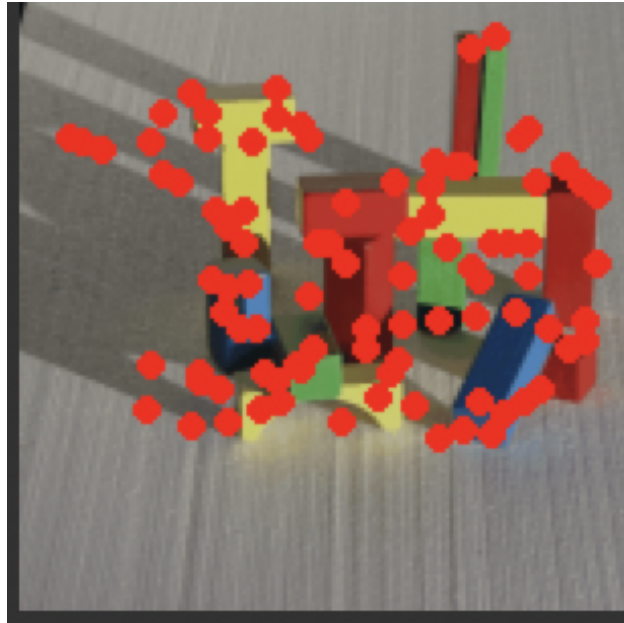


Figure 2.1. First Step: Scale-space extrema detection

2.1.2 Second Step: Keypoint localization

Using the SIFT object from earlier we can extract the second step, by applying the first step but also scaling the size of the circle that is directly proportional to the key points scale

```

1  img1KeypointsScaled = img1.copy()
2      for kp in kp1:
3          x, y = kp.pt
4          size = int(kp.size)
5          cv2.circle(img1KeypointsScaled, (int(x), int(y)), size, (0, 255, 255), 2)
6
7  imageLeftMiddlePhoto = ConvertCV2ToTKINTER(img1KeypointsScaled)
8  imageLeftMiddle.config(image=imageLeftMiddlePhoto)
9  imageLeftMiddle.image = imageLeftMiddlePhoto

```

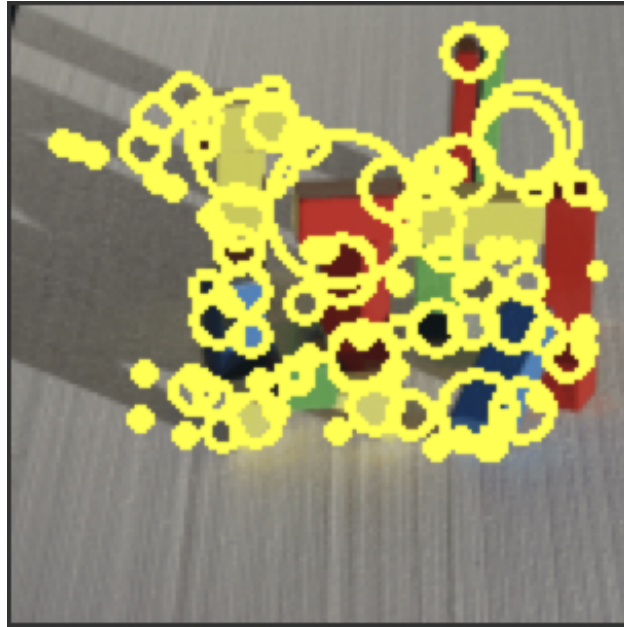


Figure 2.2. Second Step: Keypoint localization

2.1.3 Third Step: Orientation assignment

Using the SIFT object from earlier we can assign the direction and orientation of the keypoints using the angle parameter of the keypoint object and Opencv build in arrowedLine functionality.

```

1     img1Orientation = img1.copy()
2     for kp in kp1:
3         x, y = kp.pt
4         angle = np.deg2rad(kp.angle)
5         length = kp.size / 2
6         x2 = int(x + length * np.cos(angle))
7         y2 = int(y + length * np.sin(angle))
8         cv2.arrowedLine(img1Orientation, (int(x), int(y)), (x2, y2), (0, 255, 0),
9             ↪ 1, tipLength=0.3)
10
11 imageRightMiddlePhoto = ConvertCV2ToTKINTER(img1Orientation)
12 imageRightMiddle.config(image=imageRightMiddlePhoto)
13 imageRightMiddle.image = imageRightMiddlePhoto

```

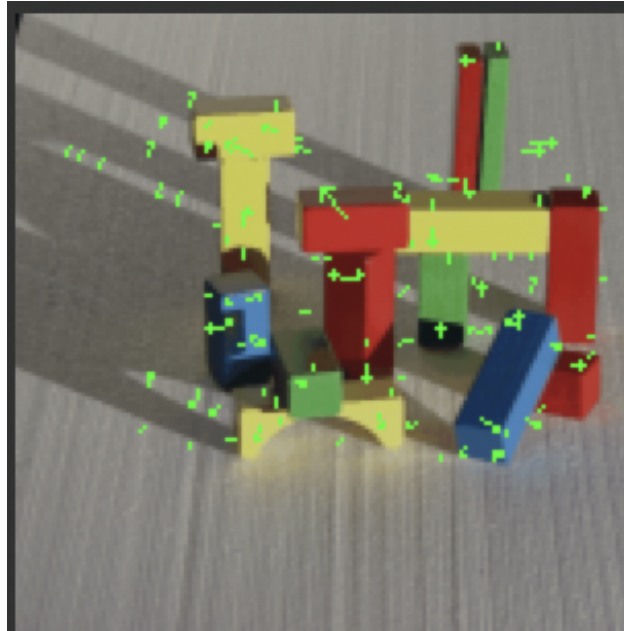


Figure 2.3. Third Step: Orientation assignment

2.1.4 Fourth Step: Keypoint descriptor

For the fourth step, this is once again a parameter given by the openCV2 and we can print the descriptor arrays these descriptors are used to then match to other images that have similar keypoint descriptors. Using BF matching we can actually attempt to match to different photos

```
1 kp1, des1 = sift.detectAndCompute(img1, None)
2 Output:
```

17	0	0	...	14	4	0
30	23	10	...	2	0	1
31	2	0	...	7	2	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮
0	2	78	...	0	0	10
68	1	1	...	3	0	2
12	112	163	...	2	0	1

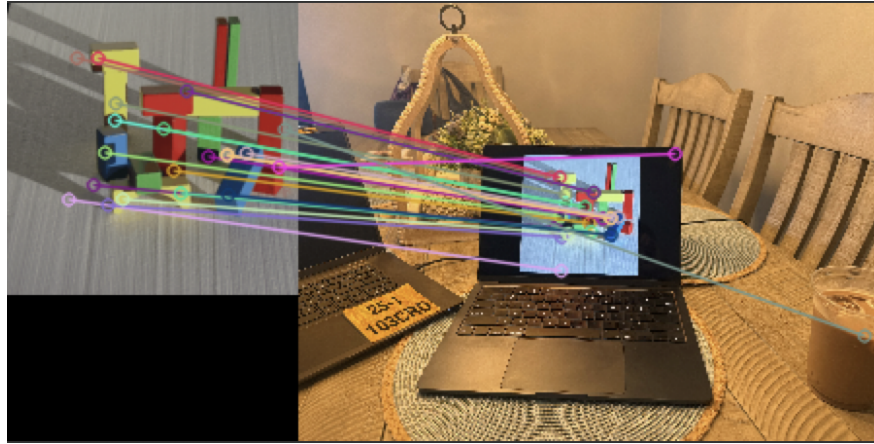


Figure 2.4. Matching

2.2 RESULTS

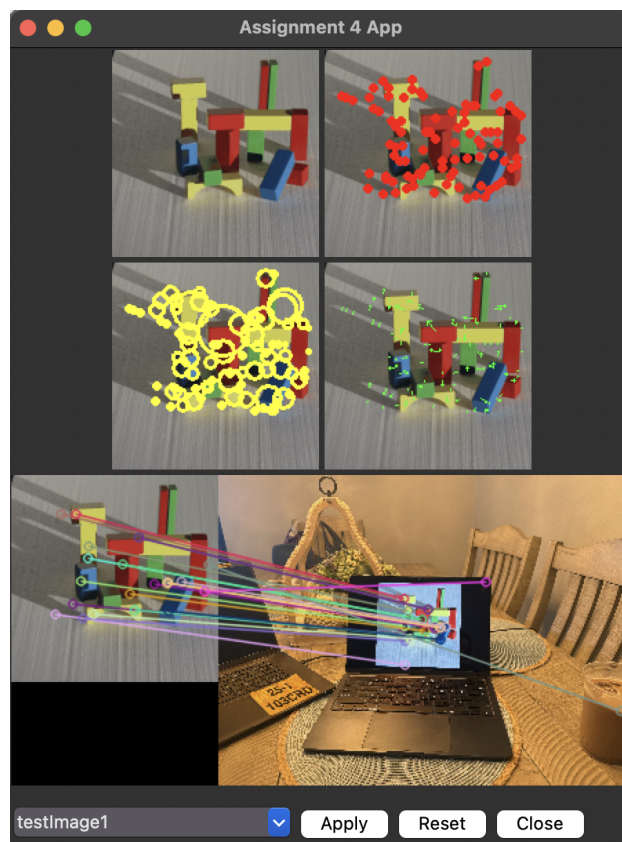


Figure 2.5. Test 1

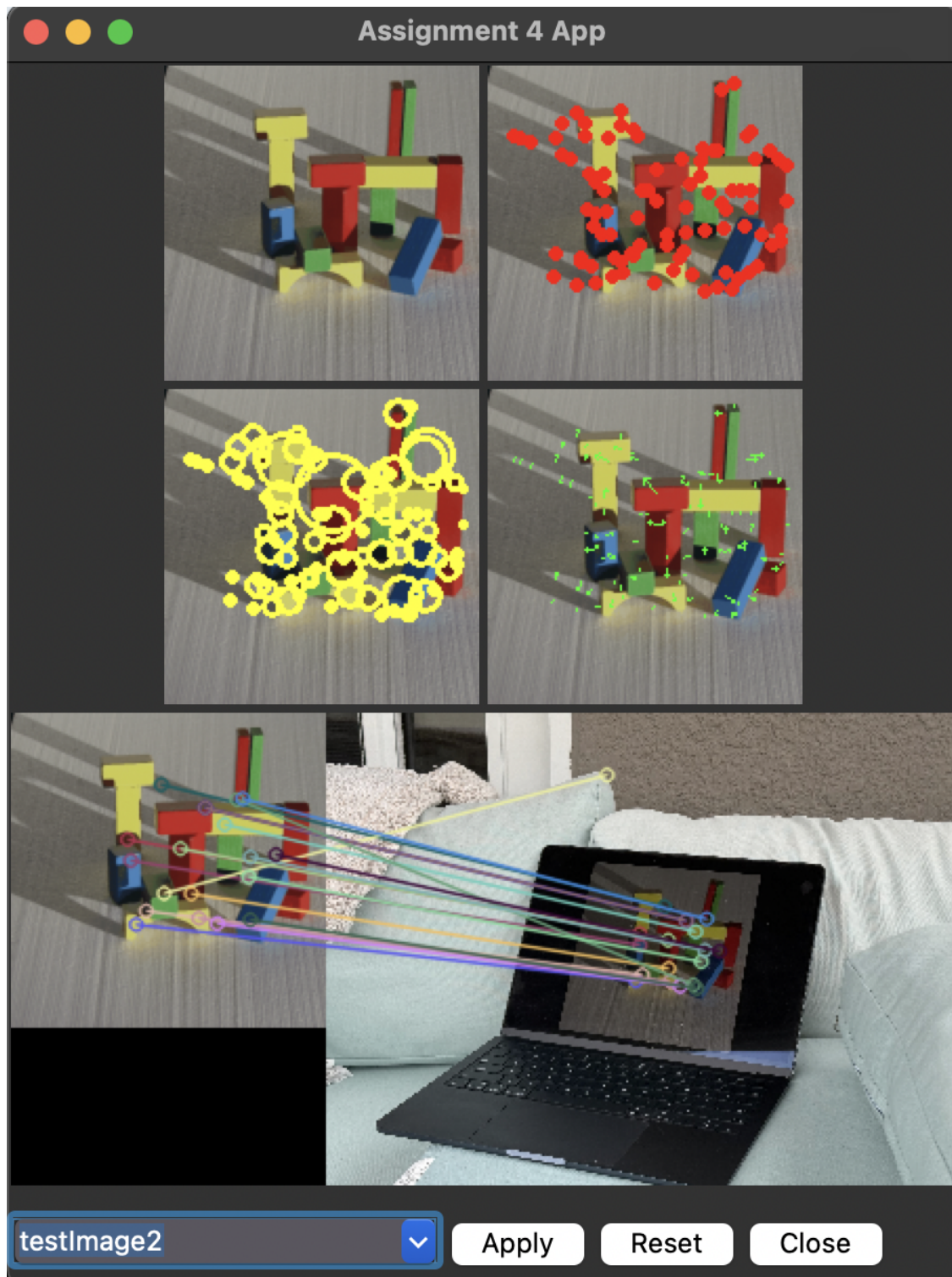


Figure 2.6. Test 2

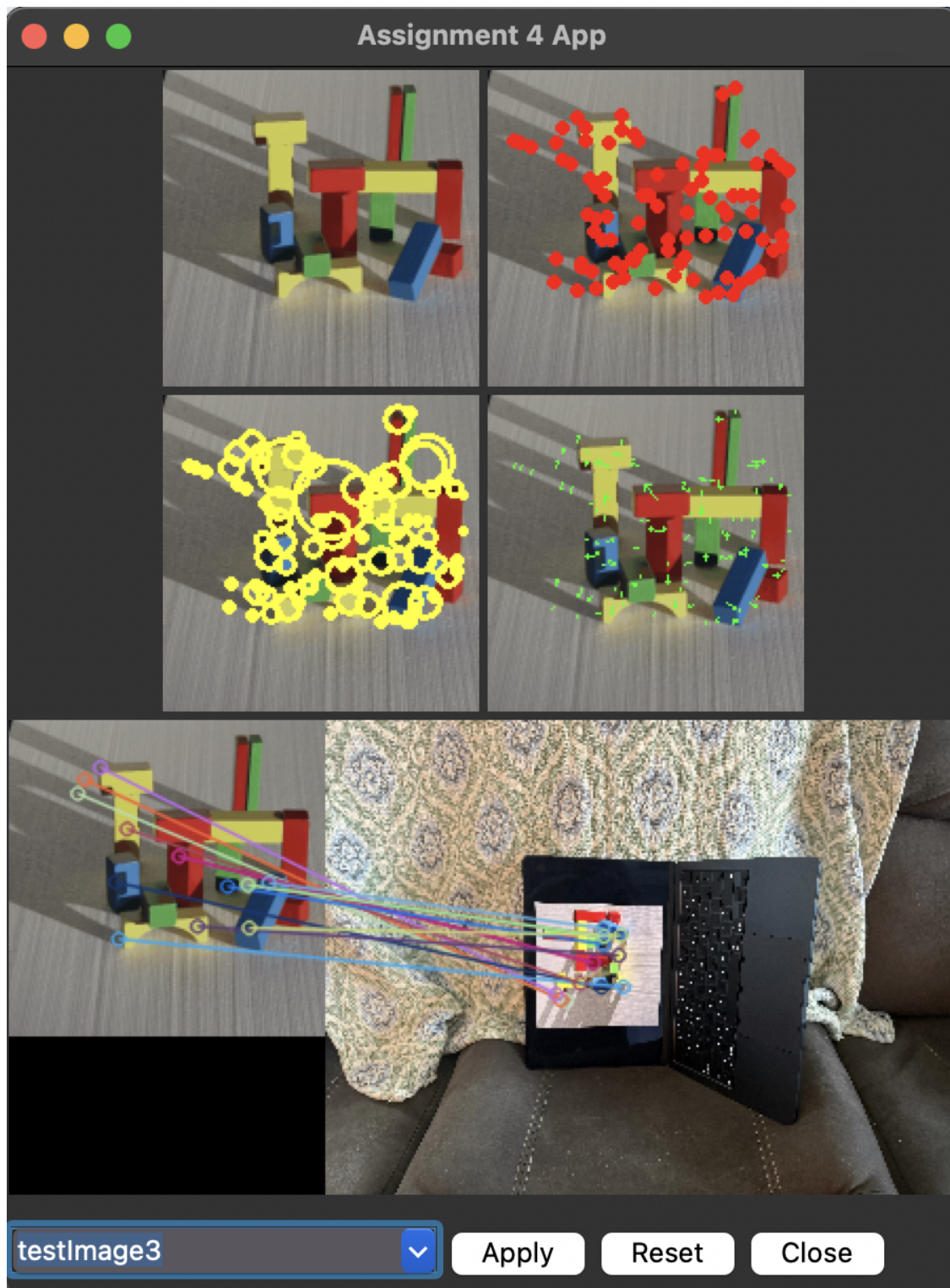


Figure 2.7. Test 2

2.3 DISCUSSION

I applied the Scale-Invariant Feature Transform (SIFT) algorithm in detecting and describing features within various versions of an image. By detecting key points in varying sizes, their locations, and their directions, I established that SIFT detects significant features even as the size of the image, direction, or illumination varies. It was found through testing that this algorithm can detect the same key points within multiple copies of the same picture, demonstrating SIFT's largest asset as its capacity for detecting features that remain consistent. Implementation revealed several significant aspects on how the algorithm functions. Building the scale-space pyramid was significant in detecting features of various sizes, and the use of the Difference of Gaussian (DoG) method worked well in approximating the Laplacian of Gaussian. The keypoint localization step enhanced the feature point quality since it discarded low-contrast points and responses of edges. Orientation assignment ensured there would be no effect due to rotation since it determined sizes of gradients as well as directions around each keypoint. The project succeeded in meeting its overall objectives but has the potential for improvement in the future. In other words, this can be made faster for real-time applications, with an additional step for improved feature matching, and the ability for varying settings on achieving the appropriate amount and quality of keypoints for various applications. In summary, this project made me understand how feature detection algorithms function and how they are applied in object recognition, image matching, as well as in building 3D models.

REFERENCES

OpenCV Documentation Team (2025a). "Feature Matching in OpenCV. Accessed: 2025-04-07.

OpenCV Documentation Team (2025b). "Introduction to SIFT (Scale-Invariant Feature Transform). Accessed: 2025-04-07.

Ron2112 (2025). "Sift - scale space extrema detection. Accessed: 2025-04-08.